

SD3589CV100
开发环境用户指南

文档版本 1.0.12

发布日期 2025-04-14

前 言

概述

本文档介绍Linux开发环境的搭建

本文档主要提供让客户更快地了解Linux开发环境指导。

版本说明

与本文档相对应的产品版本如下。

产品名称	产品版本
SD3589C	V100
SD3589D	V100

用户

本文档主要适用于以下工程师：

- ✧ 技术支持工程师
- ✧ 软件工程师
- ✧ 算法工程师

修订记录

文档版本	发布日期	修改说明
1.0.0	2023-12-22	第1次临时版本发布。
1.0.1	2024-01-18	1、 2.3章节SDK编译方法修改； 2、 快启使用说明章节以SPI-NAND为例 3、 快启使用说明章节增加sensor配置说明。
1.0.2	2024-03-08	1、更新1.2.3节软件包安装
1.0.3	2024-03-09	1、 新增SPI 7线模式使用说明

		2、 新增“7 如何修改SPI-Flash工作频率”使用说明
1.0.4	2024-03-11	1、新增“5.8 如何修改Flash在BOOTROM和OTFL中的工作频率，提高启动速度”
1.0.5	2024-03-12	1、新增编译eMMC启动镜像操作说明
1.0.6	2024-03-15	修改pip源信息
1.0.7	2024-03-17	1、新增emmc适配SPI 7线模式使用说明
1.0.8	2024-04-18	1、 变更5.5章节修改默认分区表的方式
1.0.9	2024-10-22	1、1.2.3章节步骤2中添加autopoint po4a模块安装
1.0.10	2025-4-25	新增SPI 7线DMA模式传输长度限制描述
1.0.11	2025-05-23	toolchain路径改为发布包路径
1.0.12	2025-06-04	新增SPI 7线提高TASKLET优先级配置描述

目 录

目录

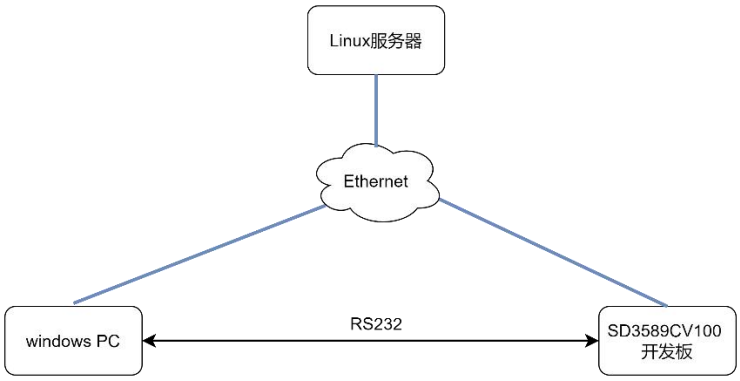
开发环境用户指南.....1
前 言.....2
1 开发环境.....1
1.1 开发组网.....1
1.2 搭建开发环境.....2
1.2.1 发布包使用的 Linux Server 版本.....2
1.2.2 网络环境搭建.....2
1.2.3 软件包安装.....2
1.2.4 安装交叉编译工具.....3
2 安装SDK.....3
2.1 下载SDK包.....3
2.2 解压缩包.....3
2.3 编译SDK包.....3
2.4 编译eMMC启动镜像.....4
2.5 编译非plusv100芯片启动镜像.....4
3 烧写SDK镜像.....4
3.1 拨码说明.....4
4 使用SDK.....5
4.1 配置demo板网络.....5
4.2 使用nfs进行开发.....5
4.3 运行sample.....5
5 快启使用说明.....6
5.1 下载解压SDK包.....6
5.2 执行编译.....6
5.3 烧写镜像.....6
5.4 更新文件系统镜像方法.....6
5.5 如何修改快启默认分区表.....6
5.6 如何修改快启拨码.....6
5.7 如何修改sensor配置.....7
5.8 如何修改Flash在BOOTROM和OTFL中的工作频率，提高启动速度.....7
6 SPI 7线模式使用说明.....8
6.1 SDK代码修改.....8
6.2 执行编译.....9
6.3 烧写镜像.....9
7 如何修改SPI-Flash工作频率.....10
8 RTC功能开始说明.....12

1 开发环境

1.1 开发组网

SD3589CV100开发环境通常包括Linux服务器、Windows和SD3589CV100DEMO BOARD（目标板），三者同处于一个网络中，如图1-2所示

图 1-2 SD3589CV100 开发环境



在Linux服务器上建立交叉编译环境，Windows通过串口和网口与SD3589CV100单板连接，开发人员可以在Windows中进行程序开发和烧写或者远程登录到Linux服务器进行程序开发。各部分具体软件介绍如表1-1所示。

表 1-1 SD3589CV100 组网各部分描述

软件		描述
Windows	操作系统	Windows10。
Linux服务器	操作系统	建议为Ubuntu 20.04。建议选择完全安装。
	应用软件	ssh，nfs，arm交叉编译环境等。
SD3589CV100	引导程序	U-boot。
	操作系统	Linux内核基于Linux标准内核5.10版本移植开发，根文件系统基于busybox制作而成。
	应用软件	包含busybox常用命令。

版权所有 © 上海云集互联有限公司

1.2 搭建开发环境

推荐用户使用64位Linux服务器，不建议在32位Linux服务器开发，可能存在未知的兼容性问题。

推荐的硬件配置如下：

- CPU Intel(R) Xeon(R) CPU E5-2450 0 @ 2.10GHz 或更好CPU
- DDR: >= 32GB
- Hard disk >= 500GB
- Ethernet
- OS: Ubuntu 20.04 64bit

1.2.1 发布包使用的 Linux Server 版本

本开发包默认均以此Linux系统进行编译：

Ubuntu 20.04

1.2.2 网络环境搭建

请自行配置网络，并安装nfs， ssh等网络组件。

1.2.3 软件包安装

Ubuntu安装好后，且用户已自行配置好网络环境，则可继续如下步骤完成相关软件包的安装：

步骤1 Shell默认配置为bash

执行`sudo dpkg-reconfigure dash` 选择 no

步骤2 安装软件包

执行

```
sudo apt-get install make libc6-i386 lib32z1 lib32stdc++6 zlib1g-dev libncurses5-dev ncurses-term libncursesw5-dev g++
u-boot-tools texinfo gawk libssl-dev openssl bc p7zip-full gperf bison flex diffutils git unzip libffi-dev libtool libfreetype6
zlib1g-dev liblzo2-dev uuid-dev pkg-config automake texlive libxml2-dev autopoint po4a
```

步骤3 配置pkgconfig

执行

```
export PKG_CONFIG_PATH="$PKG_CONFIG_PATH:/usr/lib/x86_64-linux-gnu/pkgconfig"
```

步骤4 依赖 python3相关库

下载安装并执行如下指令：

```
sudo apt install python3-pip  python-is-python3
```

可能需要配置下载源：请创建并配置pip源，编辑:vim ~/.pip/pip.conf，修改如下：

```
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/simple
```

完成后安装如下库：

```
sudo pip3 install wheel==0.36.2
```

```
sudo pip3 install pycryptodome==3.9.8
sudo pip3 install pyelftools==0.27
sudo pip3 install cryptography
```

1.2.4 安装交叉编译工具

须知

使用从网络等渠道得到的交叉编译工具可能存在与使用的内核并不配套，造成开发过程中出现一些不可预料的问题。

发布包提供编译工具链aarch64-otv02-linux-glibc-(路径位于SDK包software/pc/ot_toolchain下）。aarch64-otv02-linux-glibc-为基于64bit操作系统glibc的工具链。

安装步骤如下：**步骤1**

解压工具链。

执行如下命令进行解压：

```
tar xzf gcc-aarch64-otv02-linux-gnu.tgz
```

步骤2 安装工具链。

运行如下命令完成此工具链的安装。

```
cd gcc-aarch64-otv02-linux-gnu
sudo ./install_gcc_toolchain.sh
    source /etc/profile
```

2 安装SDK

2.1 下载SDK包

请客户从本司FAE申请软件SDK包。其中

SD3589CV100_SDK_V100R001C0[X]SPC[XYZ].tar.gz文件，就是SD3589CV100系列芯片的软件开发包。

2.2 解压缩包

在Linux服务器上，创建客户自有目录，拷贝SDK包到当前目录，并执行

```
tar -zxvf SD3589CV100_SDK_V100R001C0[X]SPC[XYZ].tar.gz
```

解压软件包到当前目录下

2.3 编译SDK包

在步骤2.2中创建的目录下，执行

```
cd lq560v100_sdk
./build.sh （快启版本执行： ./build.sh -q）
```

编译完成后会在main/pub下生成lq560plusv100_image_glibc(快启版本为

lq560plusv100_mini_image_glibc)目录，目录下为版本可烧写镜像文件。

注意：如需启用快启出图功能，请参考《SD3589CV100 Sensor 快启配置指南.docx》。

2.4 编译eMMC启动镜像

为支持eMMC启动，需要修改启动介质，具体对应代码：

1. 文件路径：main/src/platform/Makefile，将BOOT_MEDIA ?= emmc一行放开注释，同时注释掉上一行的BOOT_MEDIA ?= spi
2. 修改后执行全编译，编译完成后会在main/pub下生成lq560plusv100_emmc_image_glibc(快启版本为lq560plusv100_emmc_mini_image_glibc)目录，目录下为可烧写镜像文件。

注意：

1） 其他章节在介绍编译镜像时都以SPI-Flash为例，如果是eMMC相关镜像，请自行切换对应目录。

2） 如需启用快启出图功能，请参考《SD3589CV100 Sensor 快启配置指南.docx》。

2.5 编译非plusv100芯片启动镜像

SDK默认编译SD3589CV100镜像，编译为其他档位的芯片镜像参考readme.txt第一章节内容《切换芯片类型》

3 烧写SDK镜像

参考《SD3589CV100 OTBurnTool烧写工具指南》进行烧写。

3.1 拨码说明

启动模式	S6600(1-2-3-4)	S6601(1-2-3-4)
spi-nand 串口启动	X011	1111
spi-nand flash 一线启动	0011	1011
spi-nand flash 四线启动	1011	1011
spi-nor 串口启动	X011	0111
spi-nor flash	X011	X011
emmc 串口启动	X111	1111
emmc 4bit启动	0111	1011
emmc 8bit启动	1111	1011

*其中，0对应ON，1对应OFF，X表示当前拨码对启动模式无影响，拨码位置请见下图

 版权所有 © 上海云集互联有限公司

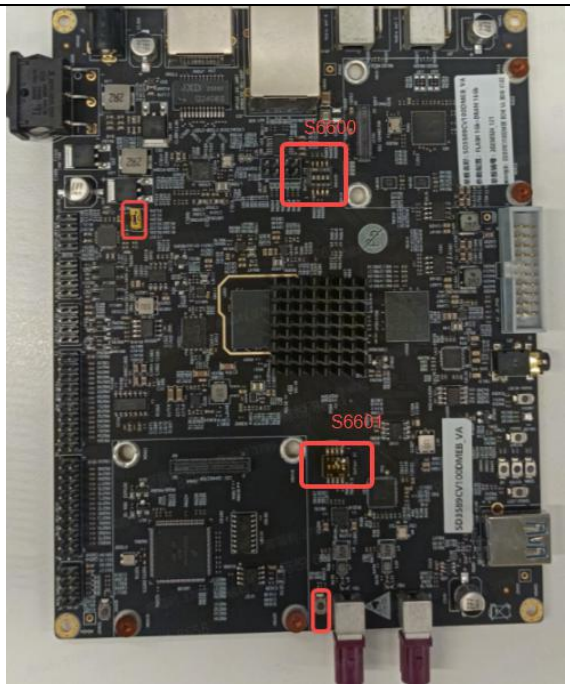


图3-1 SD3589CV100 拨码位置示意图

4 使用SDK

4.1 配置demo板网络

步骤1 设置网络

```
ifconfig eth0 hw ether xx:xx:xx:xx:xx:xx;
ifconfig eth0 xx.xx.xx.xx netmask xx.xx.xx.xx;
route add default gw xx.xx.xx.xx
```

步骤2 然后ping一下PC IP地址，如无意外，网络会正常工作

4.2 使用nfs进行开发

步骤1 在开发阶段，推荐使用NFS作为开发环境，可以省去重新制作和烧写根文件系统的工作；需确认linux服务器和Demo板网络环境已经互通

步骤2 执行挂载NFS文件系统的命令

```
mount -t nfs -o nolock -o tcp xx.xx.xx.xx:/path_to_sdk /mnt
```

命令中xx.xx.xx.xx为linux服务器IP地址

步骤3 挂载完成后可在/mnt目录下看到对应的SDK文件

4.3 运行sample

请客户首先保证Demo板对应的sensor和屏幕相关连线都已经连接好。

在Demo板下，进入/mnt/main/src/component/media/sample/vi目录。执行：

```
./sample_vi 0
```

在Demo板对应屏幕上可看到sensor采集的图像

5 快启使用说明

5.1 下载解压SDK包

参照第二章2.1,2.2章节进行下载解压

5.2 执行编译

参照第二章2.3章节进行编译

编译完成后会在main/pub下生成lq560plusv100_mini_image_glibc目录，目录下为可烧写镜像文件。

5.3 烧写镜像

以启动介质SPI-NAND为例：

参照第3章说明烧写lq560plusv100_mini_image_glibc目录下镜像；

分区表文件选择 nand_burn_table.xml

烧写镜像后，单板启动前请连接相关快启Sensor和显示屏幕，系统会自动运行媒体通路并显示实时图像。

5.4 更新文件系统镜像方法

以启动介质SPI-NAND为例：

1. rootfs更新

用户可拷贝自有文件到main/src/platform/fs/rootfs/base/目录下， 执行

make rootfs_build

重新生成rootfs_lq560plusv100.ramdisk.gz并进行烧写

2. userfs更新

用户可拷贝自有文件到 main/src/platform/fs/userfs/base 目录下 ，执行

make userfs_build

重新生成user_lq560plusv100_2k_128k_56M.ubifs并进行烧写

5.5 如何修改快启默认分区表

以非快启时启动介质SPI-NAND为例：

分区表文件：main/tools/pc/partition_env/config/lq560plusv100/nand.xml

Nandflash大小一共128Mbyte，用户可根据需要进行扩充和缩减（具体参考文档《SD3589CV100 AB分区启动用户指南.pdf》）

Attention：

改变分区大小后建议执行**全片擦除**，否则可能挂载失败；

系统默认rootfs(ramdisk)分区的内存大小设置为：initrd=0x44000000,0x2000000

如果修改rootfs，导致rootfs压缩前大小超出0x2000000，则需修改默认initrd的大小。

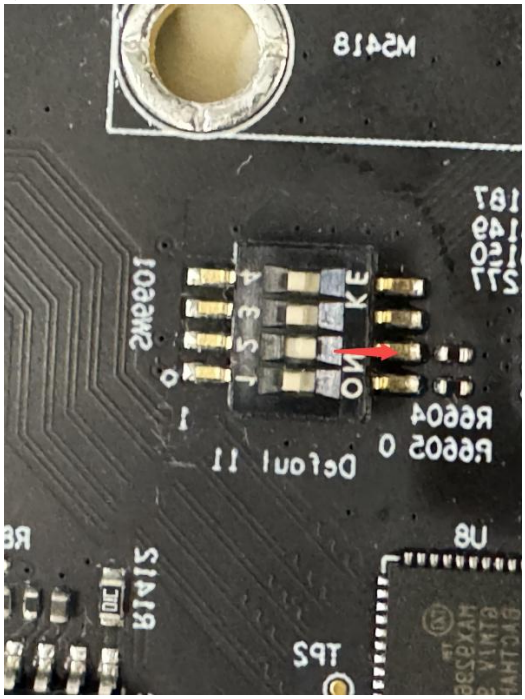
5.6 如何修改快启拨码

将拨码改为 4 线+flash 启动可以达到最好的启动速度。

详细参考<< SD3589CV100DMEB_SCH_VA.pdf>>文档

1. 将启动拨码开关拨为flash启动

拨码盘6601 开关2 拨码为on



2. 将spi_nand读取速率拨码为4线读取

对应SW6601拨码盘上 1 为"off"档位，SW6600拨码盘上 2 为 "on" 档位，1 为 "off" 档位。

注意：如果需要烧写，需要将拨码再拨回至 uart 启动，烧写完后再拨回 flash 启动（uart 启动模式也能正常启动，但是会多几十毫秒的 uart 超时时间）

5.7 如何修改sensor配置

SDK默认的sensor配置为单sensor IMX623,如果需要修改sensor配置，比如配置其他sensor或者配置为双sensor模式等，可以参考文档《SD3589CV100 Sensor 快启配置指南.docx》

5.8 如何修改Flash在BOOTROM和OTFL中的工作频率，提高启动速度

在 main/tools/pc/image_tool/oem/quick_build_config.json 中找到 OTFL_Info_area，如下

```
"OTFL_Info_area": {
    "OTFL_code_version_ext": "0x00000000",
    "mask_OTFL_code_version_ext": "0x00000000",
    "OTFL_code_enc_flag": "0x3C7896E1",
    "OTFL_code_FlashReadClk": "0x0"
},
```

OTFL_code_FlashReadClk 默认值为 0x0，将其修改为 0x1，FMC 的工作频率将从 12MHZ 提高到 75MHZ，读命令也将从 0x03 变成 Fast Read 命令 0x0b。该修改能够加快 Flash 启动的速度。若要修改 OTFL_code_FlashReadClk 的默认值，需先根据 Flash 数据手册 AC Electrical Characteristics 章节确定 Flash 是否能工作在 75MHZ 的 Fast Read 模式下。

6 SPI 7线模式使用说明

6.1 SDK代码修改

使用SPI 7线模式需修改以下代码：（NOTE: DMA模式限制传输长度为：不能大于240字节+8bit数据位宽或480字节+16bit数据位宽）

1. 文件路径：main/src/component/driver/sys_config/src/pin_mux.h
   把该宏配置为1：#define SPI2_7LINE_EN 1
2. 文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/boot/dts/vendor/lq560plusv100-demb-flash.dts

把该设备树中以下节点配置修改为下面标红的配置(NOTE:GPIO_8_5 和 GPIO_8_6 与 UART3 是复用关系，如果使用其他 GPIO，可以不用关闭 UART3）。

```
&uart3 {
    status = "disabled";
};

&spi_bus2 {
    status = "okay";
    vendor,slave_mode = <1>;
    slave-ready-gpios = <&gpio_chip8 5 GPIO_ACTIVE_LOW>;
    master-request-gpios = <&gpio_chip8 6 GPIO_ACTIVE_LOW>;

    slave@0 {
        compatible = "rohm,dh2228fv";
        reg = <0>;
        pl022,interface = <0>;
        pl022,com-mode = <0>; /* 默认 0：中断 2：DMA，如要用 DMA 请修改为 2*/
        spi-max-frequency = <8250000>;
    };
};
```

3. 仅适用 DMA 模式，文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/boot/dts/vendor/lq560v100plus-demb.dts

修改文件中如下代码段为标红的配置：

```
        &edmacv310_0 {
            status = "okay";
    };
```

4. 仅适用 DMA 模式，文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/boot/dts/vendor/lq560v100plus.dtsi
   修改文件中如下代码段为标红的配置：

```
        spi_bus2: spi@11072000 {
                compatible = "arm,pl022", "arm,primecell";
                arm,primecell-periphid = <0x00800022>;
```

```
                reg = <0x11072000 0x1000>;
                interrupts = <GIC_SPI 28 IRQ_TYPE_LEVEL_HIGH>;
                clocks = <&clock LQ560V100PLUS_SPI2_CLK>;
                clock-names = "apb_pclk";
                resets = <&clock 0x4490 0>;
                reset-names = "bsp_spi_rst";
                #address-cells = <1>;
                vendor,slave_mode = <0>;
                #size-cells = <0>;
                status = "disabled";
                num-cs = <1>;
                dmas = <&edmacv310_0 16 16>, <&edmacv310_0 17 17>;
                dma-names = "rx","tx";
            };
```

5. nand 非快启版本文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/configs/lq560plusv100_defconfig
   仅适用 DMA 模式：CONFIG_EDMACV310=y
6. nand 快启版本文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/configs/lq560plusv100_mini_defconfig
   在该文件中修改 SPI 驱动宏开关为编译为模块：CONFIG_SPI_PL022=m
   仅适用 DMA 模式：CONFIG_EDMACV310=y
7. emmc 非快启版本文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/configs/lq560plusv100_emmc_defconfig
   仅适用 DMA 模式：CONFIG_EDMACV310=y
8. emmc 快启版本文件路径：
   main/src/platform/linux/linux-5.10.y/arch/arm64/configs/lq560plusv100_emmc_mini_defconfig
   在该文件中修改 SPI 驱动宏开关为编译为模块：CONFIG_SPI_PL022=m
   仅适用 DMA 模式：CONFIG_EDMACV310=y
9. 如果要提高 SPI 和 DMA 的 TASKLET 优先级，需修改：
   文件路径：main/src/platform/linux/linux-5.10.y/drivers/spi/spi-pl022.c 中所有 tasklet_schedule(&pl022->pump_transfers);改为 tasklet_hi_schedule(&pl022->pump_transfers);

   文件路径：main/src/platform/linux/linux-5.10.y/drivers/dma/virt-dma.h 的 static inline void vchan_cookie_complete(struct virt_dma_desc *vd)函数中的 tasklet_schedule(&vc->task);改为 tasklet_hi_schedule(&vc->task);

6.2 执行编译

参照第二章2.3章节进行编译

```
cd lq560v100_sdk
./build.sh （快启版本执行： ./build.sh -q）
```

编译完成后会在main/pub下生成快启为lq560plusv100_mini_image_glibc目录，非快启为lq560plusv100_image_glibc目录，目录下为可烧写镜像文件。

6.3 烧写镜像

以启动介质SPI-NAND为例：

参照第3章说明烧写快启版本lq560plusv100_mini_image_glibc目录或非快启版本lq560plusv100_image_glibc目录下镜像；

分区表文件选择 nand_burn_table.xml

7 如何修改SPI-Flash工作频率

SDK的SPI-NOR Flash和SPI-NAND Flash工作频率可以根据需要进行修改，u-boot中对应的修改文件路径分别为：

SPI-NOR Flash：

```
main/src/platform/u-boot/u-boot-2022.07/drivers/mtd/spi/fmc100/fmc_spi_nor_ids.c;
```

SPI-NAND Flash：

```
main/src/platform/u-boot/u-boot-2022.07/drivers/mtd/nand/raw/fmc100/fmc_spi_nand_ids.c;
```

Linux中对应的修改文件路径分别为：

SPI-NOR Flash：

```
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/gigadevice.c;
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/macronix.c;
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/micron-st.c;
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/spansion.c;
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/winbond.c;
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/eon.c;
main/src/platform/linux/linux-5.10.y/drivers/mtd/spi-nor/xmc.c;
```

SPI-NAND Flash：

```
main/src/platform/linux/linux-5.10.y/drivers/mtd/nand/fmc100/fmc_spi_nand_ids.c。
```

u-boot中SPI-NOR Flash/SPI-NAND Flash和Linux SPI-NAND Flash修改频率的方式一样。下面以u-boot SPI-NOR Flash为例说明修改Flash工作频率的步骤。

1、查询Flash手册中AC Electrical Characteristics章节，找到Flash允许的最高工作频率，如下：

9.6 AC Electrical Characteristics(6)

DESCRIPTION	SYMBOL	ALT	SPEC			UNIT
			MIN	TYP	MAX	
Clock frequency except for Read Data (03h) instructions (3.0V-3.6V)	F_R	f_{C1}	D.C.		133	MHz
Clock frequency except for Read Data (03h) instructions(2.7V-3.0V)	F_R	f_{C2}	D.C.		104	MHz
Clock frequency for Read Data instruction (03h)	fR		D.C.		50	MHz

2、上述给出的修改文件头部定义了一系列Flash参数宏，如下

```
……
set_read_quad(1, INFINITE, 80);
set_read_quad4b(1, INFINITE, 80);
set_read_quad(1, INFINITE, 84);
```

```
set_read_quad(1, INFINITE, 104);
set_read_quad4b(1, INFINITE, 104);
set_read_quad(1, INFINITE, 108);
set_read_quad4b(1, INFINITE, 108);
set_read_quad(1, INFINITE, 112);
set_read_quad(1, INFINITE, 120);
set_read_quad(1, INFINITE, 133);
set_read_quad4b(1, INFINITE, 133);
set_write_quad_addr(0, 256, 33);
set_write_quad_addr(0, 256, 80);
set_write_quad_addr(0, 256, 104);
set_write_quad_addr4b(0, 256, 104);
set_write_quad_addr(0, 256, 108);
set_write_quad_addr4b(0, 256, 108);
set_write_quad_addr4b(0, 256, 120);
set_write_quad_addr(0, 256, 133);
set_write_quad_addr4b(0, 256, 133);
set_erase_sector_64k(0, _64K, 33);
set_erase_sector_64k(0, _64K, 55);
set_erase_sector_64k(0, _64K, 50);
set_erase_sector_64k(0, _64K, 80);
set_erase_sector_64k4b(0, _64K, 80);
set_erase_sector_64k(0, _64K, 86);
set_erase_sector_64k(0, _64K, 100);
set_erase_sector_64k(0, _64K, 104);
set_erase_sector_64k4b(0, _64K, 104);
set_erase_sector_64k(0, _64K, 108);
set_erase_sector_64k4b(0, _64K, 108);
set_erase_sector_64k4b(0, _64K, 120);
set_erase_sector_64k(0, _64K, 112);
set_erase_sector_64k(0, _64K, 120);
set_erase_sector_64k(0, _64K, 133);
set_erase_sector_64k4b(0, _64K, 133);
……
```

其中，第一个参数表示dummy个数、第二个参数对于读无效，对于写表示页大小，对于擦除表示擦除块大小，第三个参数表示Flash工作频率。根据步骤1，从中选择一个合适的工作频率。

3、找到修改文件中对应Flash的型号，如下

```
/* winbond W25Q128JVEAQ  3.3v */
    {
        "W25Q128(B/F)V", {0xEF, 0x40, 0x18}, 3, _16M, _64K, 3,
        {
            &read_std(0, INFINITE, 33), /* 33MHz */
```

```
            &read_fast(1, INFINITE, 104), /* 104MHz */
            &read_dual(1, INFINITE, 104), /* 104MHz */
            &read_quad(1, INFINITE, /* 70 */ 80), /* 80MHz */
            0
        },

        {
            &write_std(0, 256, 104), /* 104MHz */
            &write_quad(0, 256, /* 70 */ 80), /* 80MHz */
            0
        },

        {
            &erase_sector_64k(0, _64K, 104), /* 104MHz */
            0
        },
        &spi_driver_w25q256fv,
    },
```

根据选择的频率修改每一项中的第三个参数，如将write_quad(0, 256, /* 70 */ 80)修改为write_quad(0, 256, /*70*/ 104)。FMC IO接口频率支持12MHZ/50MHZ/75MHZ/100MHZ，选择最接近的工作频率可以最大限度提高Flash的读写擦速度。

Linux中，SPI-NOR Flash修改频率需要先根据Flash厂商找到对应的修改文件，再找到宏定义CONFIG_ARCH_BSP，如下

```
#ifdef CONFIG_ARCH_BSP
    { "gd25q16c", INFO(0xc84015, 0, 64 * 1024, 32,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(120) },
    { "gd25q32", INFO(0xc84016, 0, 64 * 1024,  64,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(120) },
    { "gd25q64", INFO(0xc84017, 0, 64 * 1024, 128,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(120) },
    { "gd25q128/gd25q127", INFO(0xc84018, 0, 64 * 1024, 256,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(80) },
    /* GigaDevice 1.8V */
    { "gd25lq16c", INFO(0xc86015, 0, 64 * 1024, 32,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(104) },
    { "gd25lq64", INFO(0xc86017, 0, 64 * 1024, 128,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(133) },
    { "gd25lq128", INFO(0xc86018, 0, 64 * 1024, 256,
            SECT_4K | SPI_NOR_QUAD_READ) PARAMS(gd), CLK_MHZ_2X(133) },
```

根据Flash数据手册中规定的最大工作频率修改CLK_MHZ_2X的参数即可。

8 RTC功能开始说明

SDK默认未开启RTC功能，如需要使用，请按照以下步骤修改Linux内核编译选项，开启RTC

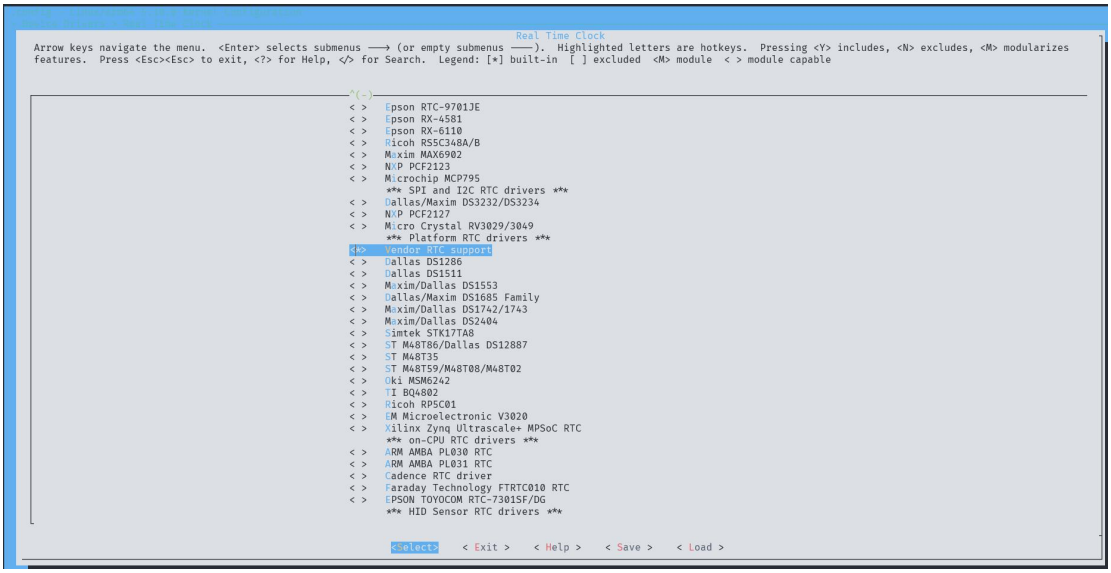
功能。需要注意：RTC功能依赖外接32K晶振，请与硬件工程师确认使用的开发板上有无外接晶振，否则RTC无法正常工作。

1.手动配置内核，在lq560v100_sdk/main/src/platform/linux/linux-5.10.y目录下执行

```
make ARCH=arm64 CROSS_COMPILE=aarch64-otv02-linux-gnu- menuconfig
```

2.开启RTC

Device Drivers ---> [*] Real Time Clock ---> <*> Vendor RTC support



3.保存并重新编译内核。